WEEK 04 : ASSINGMENT 4 AI FLUENCY

Step 1: Workflow vs Agent

An AI workflow is a predefined sequence of steps where the developer decides how the task should progress. The AI performs the tasks within that structure, but the overall process and order of actions are already defined. An agent is different because it can decide what actions to take, which tools to use, and what to do next based on the situation and the results it observes.

My FL-04 pipeline is a **workflow** not an agent. The pipeline follows a fixed three-step process: **Draft → Critique → Revise**. First, the AI creates a draft, then it critiques that draft, and finally it produces a revised version based on the critique. The next step is already determined by the pipeline, so the AI does not independently decide whether it should critique, revise, repeat the process, or stop.

Although the pipeline uses AI at multiple stages, simply using an LLM does not make a system an agent. The key difference is who controls the workflow. In FL-04, the sequence is controlled by the predefined pipeline, while an agent would have more autonomy to decide its next action.

Therefore, FL-04 is best classified as a **workflow**. It could become more agent-like if the AI were allowed to evaluate the intermediate result and decide what action to take next—for example, whether another revision is needed, whether additional information should be retrieved, or whether the output is ready to finalize.

Use the Google Drive connector to find my document named “flyrank”. Open/read the document and give me a brief summary of its Draft → Critique → Revise workflow

Used Google Drive integration, loaded tools

This is "Runs.docx" in your flyrank folder — it logs five runs of the same Draft → Critique → Revise workflow applied to your portfolio project write-ups (CRUD API, CommandDesigner, InventoryIQ, E-commerce App, Steady). Here's how the workflow works across all of them:

Step 1 — Draft
A first-pass portfolio entry is written using only the info you've given: project purpose, "my work" description, tech stack, key features, and outcome. If tech/features weren't provided yet, those fields are left as explicit placeholders rather than filled in.

Step 2 — Critique
Each draft is audited against a consistent checklist:

- **Clarity** — is it easy to follow, or does it repeat itself?
- **Accuracy** — flags whether any facts/outcomes were invented (consistently found to be clean across all 5 runs)
- **Structure** — checks if sections meaningfully add new info or just restate each other
- **Unnecessary repetition** — literally counts how many times key terms repeat
- **Unsupported claims** — checks for fabricated metrics or results
- **Weak/vague statements** — flags generic phrasing (e.g., "worked on the team," "tested successfully")
- **Contribution clarity** — the most emphasized criterion: does the entry show *your specific individual role*, not just what the project does overall

Use the Google Drive connector to find my “Runs.docx” and my “Portfolio Stack Decision & Rationale” document. Compare the two documents and explain how their purpose, process, and final output are different

Both docs live in your flyrank folder, but they serve very different purposes.

Purpose

Runs.docx is a **content-refinement log**. It's the record of five separate runs applying a writing workflow to your individual project write-ups (Internship CRUD API, CommandDesigner, InventoryIQ, E-commerce App, Steady) so each one reads well and honestly represents your contribution in your portfolio.

Portfolio Stack Decision & Rationale is a **technical decision document**. Its purpose isn't to write portfolio content — it's to decide *what to build the portfolio site with* (HTML/CSS/JS vs React+Vite vs Next.js) and justify the choice.

Process

Runs.docx follows a repeated **Draft → Critique → Revise** cycle, applied independently to each project entry:

- Draft: write the entry from given info
- Critique: audit against a fixed checklist (clarity, accuracy, repetition, unsupported claims, contribution clarity, etc.)
- Revise: produce a tightened version — or explicitly refuse to polish it further if key facts (tech stack, features) are still missing, rather than invent them

Final Output

Runs.docx ends with **multiple content deliverables**: five revised (or honestly-placeholder) project write-ups ready to drop into a portfolio, each with a tightened "My Contribution" section.

Portfolio Stack Decision ends with **one technical decision**: build the portfolio in **HTML + CSS + JavaScript, hosted on Vercel**, with the reasoning (matches current skill level, avoids learning a new framework under a two-week deadline, no backend needed) documented for future reference.

In short: Runs.docx is about *refining what your portfolio says*; the Stack Decision doc is about *choosing what your portfolio is built with*. One is iterative and content-focused, the other is a single evaluative decision that's technical and structural.

Step 2 : MCP/Connector Setup and Evidence

I connected the Google Drive connector to Claude to allow it to access documents stored outside the chat. I then ran three tasks using the connector.

Task 1 :Read and summarize a document
Claude used Google Drive to locate and read `Runs.docx` from my Flyrank folder. It retrieved the document and summarized the five runs of my Draft → Critique → Revise workflow.

Task 2 : Retrieve specific information
Claude used the Google Drive connector to find my Portfolio Stack Decision & Rationale document and identify my selected technology stack, hosting platform, and reasons for choosing it over the other options.

Task 3 :Compare two documents
Claude used Google Drive to access both documents and compare their purpose, process, and outputs.

These tasks demonstrate that the connector was actively used to retrieve external documents and information that was not available in the current chat context.

STEP 3:600–900 word explainer

Understanding AI Agents, MCP, and My FL-04 Workflow

What Is an AI Agent?

An AI agent is a system that can work toward a goal by deciding what actions to take based on the situation and the results of previous actions. Unlike a simple chatbot that mainly responds to a prompt, an agent can plan, use tools, observe the results, and decide what to do next. The important part is not simply that an AI model is being used, but that the model has some control over the process and can adapt its actions to achieve the goal.

This is different from a workflow. A workflow has a predefined sequence of steps designed by the developer. The AI may perform different tasks inside those steps, but the overall path is already established.

Workflow vs Agent: My FL-04 Pipeline

My FL-04 pipeline is a **workflow rather than an agent**. I built it around a fixed **Draft → Critique → Revise** process and applied the same structure to five portfolio-related runs:

Internship CRUD API, CommandDesigner, InventoryIQ, E-Commerce App, and the Steady UX/UI case study.

In the Draft stage, the available project information is turned into an initial portfolio entry. The Critique stage then evaluates the draft for clarity, accuracy, repetition, unsupported claims, weak statements, and contribution clarity. Finally, the Revise stage produces an improved version based on the critique.

The sequence is predetermined. The system does not independently decide whether it should critique the output, perform another revision, search for additional information, or stop. Therefore, even though an AI model performs the individual stages, the overall system remains a workflow.

What Is MCP?

The Model Context Protocol (MCP) is a standard that allows AI applications to connect with external systems and data. Instead of keeping the AI limited to information available inside the conversation, MCP provides a structured way for an AI application to interact with external capabilities.

MCP uses three important primitives: **tools, resources, and prompts**. Tools allow an AI system to perform actions, such as querying a service or executing an operation. Resources provide information or context, such as documents or other external data. Prompts are reusable templates that can guide interactions with an MCP-connected system.

The main value of MCP is therefore the connection between an AI model and external capabilities. The AI can use information or perform actions that would not be possible through ordinary chat alone.

My Connector Setup

For the practical part of this task, I connected the **Google Drive connector to Claude**. I used it to access documents stored outside the current conversation.

I ran three tasks through the connector. First, Claude located and read my `Runs.docx` file from my Flyrank folder and summarized the Draft → Critique → Revise workflow. Second, it accessed my Portfolio Stack Decision & Rationale document and retrieved specific information about my selected technology stack and the reasoning behind the decision. Third, it accessed both documents and compared their purposes, processes, and outputs.

These tasks demonstrated that Claude was retrieving external information through the connected service rather than answering only from the conversation itself.

How FL-04 Could Become an Agent

FL-04 could become more agent-like by giving the AI greater control over what happens after each stage. Instead of always following Draft → Critique → Revise exactly once, the system could evaluate the critique and decide what action is necessary.

For example, after reviewing a draft, the AI could decide whether the content is already strong enough to finalize, whether another revision is required, or whether additional information needs to be retrieved before revising. It could then repeat the process until a defined quality condition is reached.

A concrete agent upgrade for FL-04 would therefore be a **dynamic revision loop**. The AI would evaluate the revised output, decide whether another revision is needed, and determine when the final version is ready. If connected to external tools through MCP, it could also retrieve relevant project documents when information is missing instead of relying only on the original prompt.

This would move FL-04 from a fixed workflow toward an agent because the AI would have greater autonomy over the next action and would use observations from previous steps to guide its decisions.